

pgBackRest

Data Corruption Troubleshooting During Restore and Production Best Practices

Full, Differential, Incremental, Block-Level, and WAL-Based Restore Scenarios
PostgreSQL 15, 16, 17, and 18 on AlmaLinux 8/9

PostgreSQL Versions	15, 16, 17, 18
pgBackRest Version	2.58+ (current stable)
Data Directory	/apps/pgsql_data/<version>
WAL Archive	/apps/pgsql_archives/
Logs	/apps/logs
Backup Images	/apps/backups
Backup Tools	pgBackRest, pg_basebackup, pg_dump
Operating System	AlmaLinux 8 / 9

Table of Contents

1. **pgBackRest Architecture and Backup Types Overview**
2. **Corruption Scenario 1 — Full Backup Restore Failures**
3. **Corruption Scenario 2 — Differential Backup Restore Failures**
4. **Corruption Scenario 3 — Incremental Backup Restore Failures**
5. **Corruption Scenario 4 — WAL Archive Corruption During PITR**
6. **Corruption Scenario 5 — Block Incremental Restore Issues**
7. **Corruption Scenario 6 — Delta Restore Corruption**
8. **Corruption Scenario 7 — Timeline Fork and Promotion Conflicts**
9. **Corruption Scenario 8 — Repository-Level Corruption**
10. **The pgBackRest verify Command — Proactive Corruption Detection**
11. **Post-Restore Validation Procedures**
12. **pgBackRest Best Practices for PG 15/16/17/18**
13. **Configuration Reference for Production**
14. **Quick Reference — Troubleshooting Decision Tree**

1. pgBackRest Architecture and Backup Types Overview

Backup Types and Their Dependency Chains

Understanding backup dependencies is critical for troubleshooting corruption during restore. Each backup type has a different chain of dependencies that must all be intact for a successful restore.

Backup Type	What It Copies	Dependencies for Restore	Corruption Impact
Full	Entire PGDATA cluster	Self-contained — no dependencies	Isolated — only affects this backup
Differential	Files changed since last FULL	Requires the referenced FULL backup	If FULL is corrupt, diff is useless
Incremental	Files changed since last backup (any type)	Entire chain back to FULL: all prior incr + diff + full	Any corrupt link breaks entire chain
Block Incremental (PG 13+)	Changed BLOCKS within files (not whole files)	Block map + prior backups for unchanged blocks	Block map corruption prevents file reconstruction

WAL Archive in the Restore Process

Every pgBackRest restore requires WAL replay to reach a consistent state. Even a full backup restore needs the WAL files generated during the backup window. For PITR, the WAL chain extends to the target point. pgBackRest configures the `restore_command` automatically during restore.

```
# pgBackRest restore flow:
#
# 1. Reconstruct PGDATA from backup set (full + diff/incr chain)
# 2. Write recovery configuration (restore_command, recovery target)
# 3. PostgreSQL starts and replays WAL from the archive
# 4. Recovery completes at the target point
#
# If ANY step fails – backup corruption, missing WAL, or WAL corruption
# – the restore fails or recovers to an incomplete state.
```

pgBackRest Checksum Validation

pgBackRest calculates SHA-1 checksums for every file during backup and validates them during restore. When PostgreSQL data checksums are enabled (default in PG 18), pgBackRest also validates page-level checksums during backup. This provides two layers of corruption detection:

- **File-level checksums:** Every file in the backup manifest has a SHA-1 checksum verified on restore
- **Page-level checksums (when enabled):** Each 8 KB data page is validated against PostgreSQL's built-in checksum. pgBackRest logs warnings for checksum failures but does NOT abort the backup — this allows you to detect corruption early while preserving the backup.
- **PG 18 change:** Data checksums are now enabled by default at `initdb` time. All new PG 18 clusters automatically benefit from page-level validation during pgBackRest backups.

2. Corruption Scenario 1 — Full Backup Restore Failures

SCENARIO: A full backup restore fails with checksum mismatch errors, missing files, or the restored database starts but contains corrupt pages. The full backup in the repository is damaged.

Symptoms

```
# During restore:
ERROR: [028]: backup file pg1/base/16384/16385 checksum mismatch
expected 'abc123...' but found 'def456...'

# OR: after restore, PostgreSQL reports page checksum failures:
WARNING: page verification failed: calculated checksum 12345
but expected 67890 in file base/16384/16385, block 128
```

Root Causes

- **Storage corruption:** Disk errors, silent bit rot on the repository storage
- **Network corruption:** Data corrupted during transfer from DB server to repo (rare with TLS)
- **Repository filesystem issues:** NFS timeouts, incomplete writes, filesystem bugs
- **Interrupted backup:** Backup process killed mid-write; resume picked up corrupt state
- **Encryption key mismatch:** Wrong cipher key used for restore (if encryption enabled)

Troubleshooting Steps

Step 1: Verify the backup integrity

Verify Backup Integrity:

```
# Run pgBackRest verify to check all backup files and WAL in the repo
pgbackrest --stanza=mydb verify

# Check a specific backup set
pgbackrest --stanza=mydb info --set=20251118-020000F

# JSON output for programmatic analysis
pgbackrest --stanza=mydb info --output=json | python3 -m json.tool
```

Step 2: Attempt restore from a different full backup

Restore from Alternate Full Backup:

```
# List all available backups
pgbackrest --stanza=mydb info

# Restore a specific (older) full backup instead
pgbackrest --stanza=mydb restore \
--set=20251111-020000F \
--pg1-path=/apps/postgresql_data/16 \
--process-max=4

# If the older full backup works, use PITR to replay WAL forward
# to recover as much data as possible
```

Step 3: If repository is on S3/GCS/Azure with versioning

Repository Time Travel (Versioned Storage):

```
# Use --repo-target-time to read the repository as it was at a point
# before the corruption occurred (requires versioned object storage)
pgbackrest --stanza=mydb --repo=1 \
--repo-target-time='2025-11-15 00:00:00+00' \
info

# Then restore from that point-in-time view of the repository
pgbackrest --stanza=mydb --repo=1 \
--repo-target-time='2025-11-15 00:00:00+00' \
restore --pg1-path=/apps/pgsql_data/16
```

Step 4: Use a secondary repository

Restore from Secondary Repository:

```
# If you have multiple repositories configured (best practice!),
# try restoring from the other repository
pgbackrest --stanza=mydb --repo=2 restore \
--pg1-path=/apps/pgsql_data/16 \
--process-max=4
```

TIP: Always configure at least two repositories (e.g., local + S3 or local + NFS). This is the single most effective defense against repository-level corruption.

3. Corruption Scenario 2 — Differential Backup Restore Failures

SCENARIO: A differential backup restore fails because either the differential itself is corrupt, or the referenced full backup it depends on is corrupt. pgBackRest needs both to reconstruct the cluster.

Understanding the Dependency

```
# Check which full backup a differential depends on
pgbackrest --stanza=mydb info --set=20251118-020000F_20251120-020000D

# The 'backup reference total' field shows the dependency chain
# Example output:
# backup reference total: 1 full
# This diff depends on full backup 20251118-020000F
```

Troubleshooting Steps

Case A: The differential itself is corrupt

Bypass Corrupt Differential:

```
# Option 1: Restore the full backup + replay WAL past the diff point
pgbackrest --stanza=mydb restore \
--set=20251118-020000F \
--type=time \
--target='2025-11-20 23:59:59+00' \
--pg1-path=/apps/pgsql_data/16

# This restores the full backup and replays WAL through the time window
# that the differential would have covered – effectively bypassing it
```

Case B: The referenced full backup is corrupt

Corrupt Full — Fallback to Older Full:

```
# If the FULL is corrupt, ALL differentials depending on it are unusable.
# You must find an older full backup that is intact.

# List all full backups
pgbackrest --stanza=mydb info --output=json | \
python3 -c "import json,sys; d=json.load(sys.stdin)
for b in d[0]['backup']:
if b['type']=='full': print(b['label'], b['timestamp']['start'])"

# Restore the last known-good full backup + replay ALL available WAL
pgbackrest --stanza=mydb restore \
--set=<older_full_backup_label> \
--pg1-path=/apps/pgsql_data/16
```

CRITICAL: If the only full backup is corrupt and no older full exists, you cannot restore from pgBackRest. Fall back to pg_basebackup images in /apps/backups or a standby server if available. This is why retention of multiple full backups is essential.

4. Corruption Scenario 3 — Incremental Backup Restore Failures

SCENARIO: An incremental backup restore fails. Incremental backups have the longest dependency chains — every prior incremental, plus the differential (if any), plus the full, must all be intact.

Understanding the Chain

```
# Incremental restore chain example:
#
# FULL (Sun) --> INCR (Mon) --> INCR (Tue) --> INCR (Wed) --> INCR (Thu)
#
# To restore Thursday's incremental, ALL of these must be valid:
# - Sunday's FULL
# - Monday's INCR
# - Tuesday's INCR
# - Wednesday's INCR
# - Thursday's INCR
#
# If Tuesday's INCR is corrupt, you CANNOT restore Wed or Thu.

# Check the full dependency chain
pgbackrest --stanza=mydb info --set=<incr_label>
# 'backup reference total' shows how many backups are needed
```

Troubleshooting Steps

Incremental Chain Recovery:

```
# Step 1: Identify which backup in the chain is corrupt
pgbackrest --stanza=mydb verify

# Step 2: Restore the last good backup BEFORE the corrupt one
# Example: If Tuesday's INCR is corrupt, restore Monday's INCR
pgbackrest --stanza=mydb restore \
--set=<monday_incr_label> \
--pg1-path=/apps/pgsql_data/16

# Step 3: Use PITR to replay WAL past the corruption gap
# This recovers data from Tue/Wed/Thu via WAL replay instead
pgbackrest --stanza=mydb restore \
--set=<monday_incr_label> \
--type=time \
--target='2025-11-21 23:59:59+00' \
--pg1-path=/apps/pgsql_data/16
```

TIP: To reduce incremental chain risk, use a **weekly full + daily differential** strategy instead of long incremental chains. A differential only depends on the most recent full, not on every prior backup.

5. Corruption Scenario 4 — WAL Archive Corruption During PITR

SCENARIO: The backup restores successfully, but PITR fails because one or more WAL segments in the archive are corrupt or missing. Recovery stops at the last valid WAL position.

Symptoms

```
# PostgreSQL log during recovery:
LOG: restored log file "00000001000000000000000005" from archive
LOG: restored log file "00000001000000000000000006" from archive
ERROR: could not restore file "00000001000000000000000007" from archive
FATAL: the database system is in recovery mode – cannot proceed

# OR: WAL segment exists but is corrupt:
FATAL: could not read WAL record at 0/7000A30: incorrect resource manager
data checksum in record at 0/7000A30
```

Troubleshooting Steps

WAL Archive Troubleshooting:

```
# Step 1: Check which WAL segments exist in the archive
pgbackrest --stanza=mydb repo-ls archive/mydb/16-1/ | sort

# Step 2: Identify gaps
pgbackrest --stanza=mydb verify
# verify will report missing or corrupt WAL segments

# Step 3: If the WAL gap is small, try recovering to just before it
pgbackrest --stanza=mydb restore \
--type=lsn \
--target='0/6FFFFFF0' \
--pg1-path=/apps/pgsql_data/16

# Step 4: Check if the missing WAL exists in pg_wal on the DB server
ls /apps/pgsql_data/16/pg_wal/00000001000000000000000007

# If found, manually copy to the archive path that pgBackRest uses
# (check pgbackrest.conf for repo1-path)
```

Using archive-copy for WAL Insurance

pgbackrest.conf — archive-copy:

```
# The archive-copy option stores WAL segments directly IN the backup
# This is insurance against archive corruption
[global]
archive-copy=y

# With archive-copy, restore will use WAL from the backup's pg_wal
# directory FIRST, then fall back to the archive
# This adds storage but protects against archive-level corruption
```

WARNING: If archive-push failed silently (e.g., `archive_command` returned success but did not actually copy the file), the WAL gap is permanent. This is why monitoring `pg_stat_archiver.failed_count` is critical. A backup after the gap is needed.

6. Corruption Scenario 5 — Block Incremental Restore Issues

SCENARIO: Block incremental backups (introduced in pgBackRest 2.46+) track changes at the sub-file block level. Corruption in the block map or individual blocks can prevent file reconstruction.

How Block Incremental Works

Instead of copying entire files when they change, block incremental divides large files into blocks and only copies the blocks that changed. The block map tracks which blocks belong to which backup. On restore, pgBackRest reconstructs files by assembling blocks from multiple backups.

Troubleshooting

Block Incremental Troubleshooting:

```
# Error: block list corruption
ERROR: [028]: unable to reconstruct file from block incremental

# Step 1: Verify the backup set
pgbackrest --stanza=mydb verify

# Step 2: Restore using --set to pick a non-block-incremental backup
# If you have a full or diff backup that doesn't use block incremental,
# restore from that + WAL replay
pgbackrest --stanza=mydb restore \
--set=<full_backup_label> \
--pg1-path=/apps/pgsql_data/17

# Step 3: Consider disabling block incremental for critical environments
# In pgbackrest.conf:
# repo1-block=n
```

NOTE: Block incremental is most beneficial for very large databases (100+ GB) with small change rates. For smaller databases, the complexity may outweigh the storage savings.

7. Corruption Scenario 6 — Delta Restore Corruption

SCENARIO: A delta restore (`m('--delta')`) leaves the database in an inconsistent state because some files in the existing PGDATA were retained when they should have been replaced.

How Delta Restore Works

A normal restore requires an empty PGDATA directory. A `--delta` restore compares files in the existing PGDATA with the backup using checksums and only replaces files that differ. This is faster but carries risk if the existing PGDATA has files with matching checksums but different content (hash collision) or if extra files are present that should not be.

Symptoms

- Database starts but certain tables show old/stale data
- Index inconsistencies — query results differ between index scan and seq scan
- Unexpected files in PGDATA that are not part of the backup
- `pg_amcheck` reports corruption after delta restore

Troubleshooting Steps

Clean Restore vs. Delta:

```
# Step 1: Try a clean restore instead of delta
# Stop PostgreSQL, remove PGDATA, then restore
sudo systemctl stop postgresql-16
rm -rf /apps/pgsql_data/16/*

pgbackrest --stanza=mydb restore \
--pg1-path=/apps/pgsql_data/16 \
--process-max=4

# Step 2: If delta restore is required (e.g., large DB, minimal changes),
# delete known-problematic directories first
rm -rf /apps/pgsql_data/16/base/
rm -rf /apps/pgsql_data/16/global/
rm -rf /apps/pgsql_data/16/pg_xact/

pgbackrest --stanza=mydb restore --delta \
--pg1-path=/apps/pgsql_data/16
```

WARNING: Delta restore should NEVER be used if you suspect the PGDATA has been tampered with or corrupted by malware. Always use a clean restore in security-sensitive scenarios.

8. Corruption Scenario 7 — Timeline Fork and Promotion Conflicts

SCENARIO: Restore fails with a timeline fork error. This typically occurs when a standby was accidentally promoted or a test restore was performed without disabling WAL archiving, polluting the archive with WAL from a different timeline.

Symptoms

```
ERROR: [058]: target timeline 2 forked from backup timeline 1
at 0/99000060 which is before backup lsn of 0/9a000028
HINT: was the target timeline created by accidentally promoting a standby?
HINT: was the target timeline created by testing a restore without
--archive-mode=off?
```

Root Causes

- **Accidental standby promotion:** A standby was promoted, creating Timeline 2, and its WAL was archived to the same repository
- **Test restore without --archive-mode=off:** A test restore created a new timeline and archived WAL back to the production repo
- **Split-brain:** Two servers both believe they are primary and archive conflicting WAL

Troubleshooting Steps

Timeline Fork Resolution:

```
# Step 1: Identify the timeline situation
pgbackrest --stanza=mydb repo-ls archive/mydb/16-1/
# Look for multiple timeline directories (0000000100000000, 0000000200000000)

# Step 2: Check .history files
pgbackrest --stanza=mydb repo-get \
archive/mydb/16-1/00000002.history

# Step 3: Restore to a specific timeline
pgbackrest --stanza=mydb restore \
--target-timeline=1 \
--pg1-path=/apps/pgsql_data/16

# Step 4: Take a new full backup IMMEDIATELY after resolving
pgbackrest --stanza=mydb backup --type=full
```

CRITICAL: When testing restores, **ALWAYS** use --archive-mode=off to prevent the test cluster from archiving WAL back to the production repository. This is the most common cause of timeline fork errors.

Safe Test Restore:

```
# CORRECT way to test a restore:
pgbackrest --stanza=mydb restore \
--pg1-path=/apps/pgsql_data/16_test \
--archive-mode=off \
--type=immediate \
--target-action=promote
```

9. Corruption Scenario 8 — Repository-Level Corruption

SCENARIO: The pgBackRest repository itself (backup manifests, file bundles, or the repo directory structure) is corrupted due to storage failure, incomplete writes, or malware.

Troubleshooting Steps

Repository Corruption Recovery:

```
# Step 1: Run verify to identify all corruption in the repo
pgbackrest --stanza=mydb verify --verbose

# Step 2: Check stanza health
pgbackrest --stanza=mydb check

# Step 3: If individual backup manifests are corrupt,
# list backups and try to restore from non-corrupt ones
pgbackrest --stanza=mydb info

# Step 4: If the entire repo is corrupt, fall back to:
# a) Secondary repository (--repo=2)
# b) pg_basebackup images in /apps/backups/
# c) Standby server promotion
# d) pg_dump logical backups in /apps/backups/

# Step 5: After recovery, take a new full backup to re-establish
# a clean baseline
pgbackrest --stanza=mydb backup --type=full --repo=1
```

Protecting the Repository

- Use multiple repositories (repo1-path + repo2-path) on different storage
- Enable encryption to prevent unauthorized modification
- Use versioned object storage (S3/GCS/Azure) with object locking for ransomware protection
- Set repo filesystem as read-only for all users except pgBackRest
- Monitor repo size and file counts — sudden changes indicate problems

10. The pgBackRest verify Command — Proactive Corruption Detection

The `verify` command (pgBackRest 2.36+) proactively scans the repository for corruption without performing a restore. Run it regularly to detect problems before you need a restore.

pgBackRest verify:

```
# Basic verify – checks all backups and WAL in the repository
pgbackrest --stanza=mydb verify

# Verify with verbose output (shows individual file checks)
pgbackrest --stanza=mydb verify --verbose

# What verify checks:
# - Backup manifest existence and validity
# - File checksums match manifest
# - WAL archive continuity (no gaps)
# - WAL checksum validation
# - Backup dependency chain integrity

# Example output with corruption:
# WARN: backup '20251118-020000F' file 'pg1/base/16384/16385' checksum
# invalid (expected abc123, actual def456)
# ERROR: WAL segment '00000001000000000000000007' missing from archive
```

Automated Verify Schedule

Cron-Based Verify:

```
# Add to crontab – run verify daily during low-activity period
# /etc/cron.d/pgbackrest-verify
30 3 * * * postgres pgbackrest --stanza=mydb verify \
--log-level-console=warn \
>> /apps/logs/pgbackrest_verify.log 2>&1

# Alert on any warnings or errors
if pgbackrest --stanza=mydb verify 2>&1 | grep -qiE 'WARN|ERROR'; then
echo 'pgBackRest verify found issues!' | mail -s 'Backup Alert' dba@company.com
fi
```

11. Post-Restore Validation Procedures

After any restore — especially when troubleshooting corruption — validate the restored database thoroughly.

Post-Restore Validation:

```
# Step 1: Verify PostgreSQL started cleanly
grep -iE 'error|fatal|panic|corrupt|checksum' /apps/logs/postgresql-*.log

# Step 2: Check all databases are accessible
psql -U postgres -c "SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database;"

# Step 3: Verify data checksums (PG 12+ with pg_checksums, or pg_amcheck)
psql -U postgres -d mydb -c "CREATE EXTENSION IF NOT EXISTS amcheck;"
psql -U postgres -d mydb -c "
SELECT bt_index_check(c.oid, heapallindexed => true)
FROM pg_index i
JOIN pg_class c ON c.oid = i.indexrelid
WHERE c.relam = (SELECT oid FROM pg_am WHERE amname = 'btree');"

# Step 4: Check recovery endpoint
grep 'last completed transaction' /apps/logs/postgresql-*.log

# Step 5: Rebuild indexes if any checksum failures detected
psql -U postgres -d mydb -c "REINDEX DATABASE mydb;"

# Step 6: Run ANALYZE to update statistics
psql -U postgres -d mydb -c "ANALYZE VERBOSE;"

# Step 7: Run pg_amcheck (PG 14+) for comprehensive corruption scan
pg_amcheck --install-missing --heapallindexed -d mydb
```

NOTE: PostgreSQL 18 enables data checksums by default. For PG 15/16/17 clusters that were initialized without checksums, you can enable them offline using `{m('pg_checksums --enable -D $PGDATA')}` (PG 12+).

12. pgBackRest Best Practices for PG 15/16/17/18

Backup Strategy

Strategy	Schedule	Retention	Best For
Conservative	Weekly Full + Daily Diff	4 Full + 28 Diff	Critical production, moderate size (< 500 GB)
Balanced	Weekly Full + Daily Incr	2 Full + 14 Incr	Large databases (500 GB – 5 TB)
Storage-Optimized	Monthly Full + Weekly Diff + Daily Incr	2 Full + 8 Diff + 30 Incr	Very large databases (5+ TB) with storage constraints

Configuration Best Practices

- **Multiple repositories:** Always configure at least two repos. Local (fast restore) + remote/S3 (disaster recovery). This is the single most effective protection against repository corruption.
- **Compression:** Use `compress-type=zst` with `compress-level=3` — best balance of speed and ratio. For CPU-constrained environments, use `{m('compress-type=lz4')}` for maximum throughput.
- **Parallelism:** Set `process-max=4` for backups (25% of CPUs). For restores, use all available CPUs since PostgreSQL is offline: `process-max=8` or higher.
- **archive-copy:** Enable `archive-copy=y` to store WAL segments inside the backup. Uses more storage but protects against archive corruption.
- **Encryption:** Enable `repo1-cipher-type=aes-256-cbc` for all repositories. Store the cipher key securely (vault, KMS, not in plaintext config).
- **Backup from standby:** Use `backup-standby=y` to reduce load on the primary. Requires both primary and standby configured in `pgbackrest.conf`.
- **start-fast:** Enable `start-fast=y` to request an immediate checkpoint when backup starts, reducing the time to acquire a consistent snapshot.
- **Log retention:** Set `log-level-file=detail` and `log-level-console=info` for comprehensive logging to `/apps/logs` without excessive console output.

Version-Specific Best Practices

PostgreSQL	pgBackRest Recommendation
PG 15	Enable data checksums (<code>pg_checksums --enable</code>). Use <code>pg_amcheck</code> for corruption detection. pgBackRest validates page checksums during backup when enabled.
PG 16	Block incremental backup support is mature — consider enabling for large databases. Combined with failover slot synchronization for improved HA.
PG 17	Improved <code>pg_amcheck</code> . Use with <code>pgBackRest verify</code> for comprehensive integrity validation. Finer-grained WAL control improves PITR precision.
PG 18	Data checksums ON by default at <code>initdb</code> . pgBackRest automatically validates page checksums. Upgrading from PG 15/16/17: <code>pg_upgrade</code> preserves checksum setting from source cluster — enable manually if source did not have checksums. Run <code>pg_checksums --enable</code> on the upgraded cluster if needed.

Testing Best Practices

- **Monthly restore test:** Restore to a separate directory (`/apps/pgsql_data/_test`) with `--archive-mode=off` and verify data integrity
- **Run `pgbackrest verify` daily** via cron to detect repository corruption early
- **Test PITR:** Periodically create a named restore point (`SELECT pg_create_restore_point('test_rp')`), then verify you can restore to it
- **Monitor `pg_stat_archiver`:** Alert if `failed_count > 0` — a single archive failure breaks the WAL chain

13. Configuration Reference for Production

pgbackrest.conf — Global Section:

```
# /etc/pgbackrest/pgbackrest.conf
# Production configuration for PG 15/16/17/18

[global]
# --- Repository 1: Local (fast restore) ---
repo1-path=/apps/backups/pgbackrest
repo1-retention-full=4
repo1-retention-diff=14
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=<your-secure-key>

# --- Repository 2: Remote/S3 (disaster recovery) ---
# repo2-type=s3
# repo2-s3-bucket=my-pg-backups
# repo2-s3-region=us-east-1
# repo2-s3-endpoint=s3.amazonaws.com
# repo2-s3-key=<access-key>
# repo2-s3-key-secret=<secret-key>
# repo2-retention-full=2
# repo2-cipher-type=aes-256-cbc
# repo2-cipher-pass=<your-secure-key>

# --- Performance ---
process-max=4
compress-type=zst
compress-level=3
start-fast=y

# --- WAL Protection ---
archive-copy=y
archive-async=y
spool-path=/apps/pgsql_data/pgbackrest-spool

# --- Logging ---
log-path=/apps/logs
log-level-console=info
log-level-file=detail

# --- Delta restore by default ---
# delta=y # Uncomment if you always want delta restores
```

pgbackrest.conf — Stanza Sections:

```
# --- Stanza Configuration (one per PG cluster) ---

[pg16db]
pg1-path=/apps/pgsql_data/16
# pg2-host=standby-server # If backup-standby=y
# pg2-path=/apps/pgsql_data/16

[pg17db]
pg1-path=/apps/pgsql_data/17

[pg18db]
pg1-path=/apps/pgsql_data/18
```

postgresql.conf — Archive Settings:

```
# postgresql.conf – Required settings (all PG versions)
archive_mode = on
archive_command = 'pgbackrest --stanza=pg16db archive-push %p'
wal_level = replica

# Recommended for pgBackRest operations
log_directory = '/apps/logs'
```

14. Quick Reference — Troubleshooting Decision Tree

Use this table during an active restore failure to quickly identify the correct recovery path.

Error / Symptom	Likely Cause	Fix
Checksum mismatch during restore	Backup file corrupted in repo	Restore from alternate backup set or repo2
WAL segment not found	Gap in WAL archive (archive-push failed)	PITR to before gap; copy WAL from pg_wal if available
WAL checksum invalid	WAL file corrupted in archive	PITR to before corrupt WAL; use archive-copy if enabled
Timeline fork error	Accidental promotion or test restore polluted archive	Restore with --target-timeline=N; take new full backup
Incr/diff restore fails	A backup in the dependency chain is corrupt	Restore last good link in chain + WAL replay forward
Delta restore: stale data	Existing PGDATA files incorrectly retained	Clean restore (remove PGDATA first)
Page checksum failure after restore	Source DB had corruption before backup	REINDEX; use zero_damaged_pages for extraction; restore older backup
Block incremental reconstruction error	Block map or block data corrupt	Restore from full/diff backup instead
Encryption key error	Wrong cipher key in config	Verify repo-cipher-pass matches backup time key
Repository manifest corrupt	Storage failure on repo host	Switch to repo2; verify repo storage health

Emergency Recovery Priority Order

1. Try restoring from the most recent backup in the primary repository
2. If repo1 corrupt, switch to repo2 (secondary repository)
3. If specific backup corrupt, try an older backup + WAL replay (PITR)
4. If WAL archive has gaps, use --type=immediate for earliest consistent point
5. Fall back to pg_basebackup images in /apps/backups/ + manual WAL replay
6. Use standby server (promote or copy its PGDATA)
7. Fall back to pg_dump logical backups in /apps/backups/
8. Last resort: pg_resetwal on damaged PGDATA + extract via pg_dump

End of Guide